



CIRCUITOS DIGITALES Y MICROCONTROLADORES 2026

Facultad de Ingeniería
UNLP

Interrupciones
concepto y programación

Ing. José Juárez

Introducción

- La CPU de un microcontrolador ejecuta instrucciones secuencialmente.
- Sin embargo, las aplicaciones requieren del uso de diferentes periféricos (internos o externos) y por lo tanto la CPU debe contar con un mecanismo para interactuar con ellos y dar respuesta adecuada a sus demandas.
- Los periféricos generalmente requieren la atención de la CPU de manera aleatoria en respuesta a algún evento.

Por ej: “se recibió un byte desde la interfaz serie y requiere ser almacenado”

1) Una manera de interactuar con un periférico es utilizando el método de encuesta o polling:

Ejemplo:

- Esperar se presione una tecla (caso visto en Tp1)

Otro ejemplo:

- Esperar recepción de un byte desde la PC (utilizando interfaz serie)
- En ambos ejemplos, la CPU consumirá ciclos de reloj “en espera” a que se produzca el evento determinado sin realizar otra tarea.
- Existen técnicas para hacer uso eficiente del polling, por ejemplo las MEF.

Introducción

2) Otra manera de interactuar con un periférico es a través del mecanismo de **INTERRUPCION**

- Si un periférico requiere la atención de la CPU (para ejecutar un determinado código) éste envía un pedido de interrupción a la CPU a través de una señal de hardware (con un nivel lógico).
- Una INTERRUPCION es entonces un MECANISMO DE HARDWARE que permite suspender temporalmente la ejecución normal de un programa para brindar servicio a la solicitud requerida por un periférico.
- Si la CPU acepta la interrupción, suspende la ejecución del programa en curso y pone en ejecución una rutina de código que interacciona con dicho periférico. Esta rutina especial se denomina *“Rutina de Servicio de Interrupción o ISR en inglés”* o *“Manejador de interrupción”* y se ejecuta a través de la administración *del controlador de interrupciones*.
- Luego se debe reanudar la ejecución normal del programa en el punto donde fue suspendido.



Tabla de Vectores- AVR (ver CH12 hoja de datos)

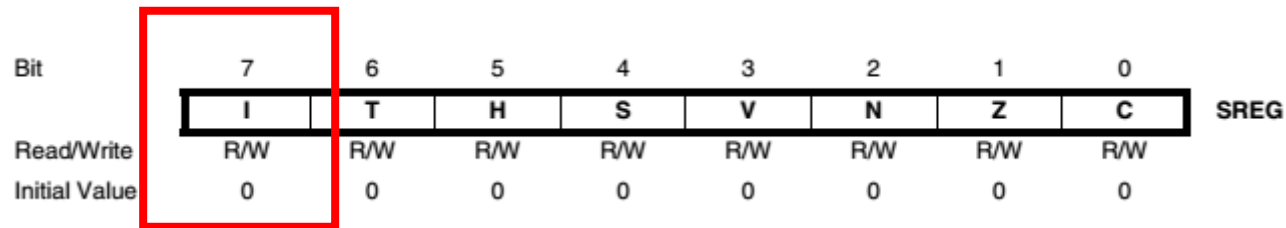
Table 11-6. Reset and Interrupt Vectors in ATmega328P

alta prioridad	VectorNo.	Program Address ⁽²⁾	Source	Interrupt Definition
	1	0x0000 ⁽¹⁾	RESET	External Pin, Power-on Reset, Brown-out Reset and Watchdog System Reset
	2	0x0002	INT0	External Interrupt Request 0
	3	0x0004	INT1	External Interrupt Request 1
	4	0x0006	PCINT0	Pin Change Interrupt Request 0
	5	0x0008	PCINT1	Pin Change Interrupt Request 1
	6	0x000A	PCINT2	Pin Change Interrupt Request 2
	7	0x000C	WDT	Watchdog Time-out Interrupt
	8	0x000E	TIMER2 COMPA	Timer/Counter2 Compare Match A
	9	0x0010	TIMER2 COMPB	Timer/Counter2 Compare Match B
	10	0x0012	TIMER2 OVF	Timer/Counter2 Overflow
	11	0x0014	TIMER1 CAPT	Timer/Counter1 Capture Event
	12	0x0016	TIMER1 COMPA	Timer/Counter1 Compare Match A
	13	0x0018	TIMER1 COMPB	Timer/Counter1 Compare Match B
	14	0x001A	TIMER1 OVF	Timer/Counter1 Overflow
	15	0x001C	TIMER0 COMPA	Timer/Counter0 Compare Match A
	16	0x001E	TIMER0 COMPB	Timer/Counter0 Compare Match B
	17	0x0020	TIMER0 OVF	Timer/Counter0 Overflow
	...	...	...	...
Baja prioridad				

- Atmega328 tiene 26 interrupciones distintas

Habilitando las Interrupciones

- El registro de Estado SREG, contiene el bit I para tal fin:



- El bit I: se llama mascara global de interrupciones y debe estar en “1” para permitir que las mismas sean atendidas por la CPU
- En assembler se utiliza la instrucción **SEI** para habilitar las interrupciones y la instrucción **CLI** para deshabilitarlas.
- En C se utilizan los macros **cli()** y **sei()**

Habilitando las Interrupciones

- La programación de los periféricos y la posibilidad de que generen interrupciones es responsabilidad del programador (en las siguientes clases veremos como configurar los distintos periféricos para que generen interrupciones).
- Para que una interrupción generada por un periférico sea aceptada, la máscara global I debe estar en 1.
- Si I=0, la CPU no acepta interrupción y la interrupción “queda pendiente a ser atendida” hasta que I=1
- A si mismo, si la CPU acepta la interrupción se debe “informar ” al periférico que fue atendida mediante alguna forma de reconocimiento de interrupción
- Cada periférico posee un *flag de estado* de interrupción y este debe ser borrado para poder procesar nuevas solicitudes.
- Esta tarea se realiza al principio de la RSI para permitir atender un nuevo pedido del mismo periférico tan pronto termine el pedido en curso (en algunos casos este reconocimiento es automático pero en otros debe hacerlo el programador).

Pasos de ejecución de una interrupción

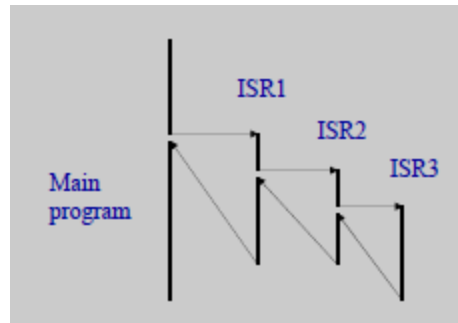
1. Cuando un pedido de interrupción es reconocido y aceptado, la CPU finaliza la instrucción que estaba ejecutando y almacena el contador de programa PC (que apunta a la siguiente instrucción) en la PILA (indicada por el SP – stack pointer).
2. Se deshabilita la máscara global I ($I=0$).
3. Se busca en la “Tabla de Vectores de Interrupción” la posición de memoria asignada a esa interrupción y se ejecuta el salto (cambio del PC) a la RSI que corresponda ejecutar.
4. LA CPU ejecuta la RSI hasta encontrar la instrucción de retorno RETI.
5. Al ejecutar RETI, el PC original que estaba en PILA es restaurado para poder continuar con la ejecución normal del programa que se estaba ejecutando antes de la interrupción y se habilita nuevamente la máscara global I.

Mecanismo de Prioridad

- La **prioridad** de una interrupción se utiliza para decidir cual interrupción debe ser atendida primero, si se diera el caso de que hay múltiples solicitudes de interrupciones al mismo tiempo.
- La prioridad esta determinada por el **orden** que aparecen los vectores en la tabla. Así las primeras son las más prioritarias y las últimas las menos prioritarias.
- **Ejemplo:** durante la ejecución de una determinada RSI se deshabilita automáticamente la mascara I ($I=0$). Al finalizar y ejecutar RETI se vuelve a habilitar ($I=1$) y se encuentra que dos periféricos han pedido servicio de interrupción: INT0 (interrupción externa 0) y UART Rx (recepción serie). Entonces el controlador de interrupciones decide que por prioridad la solicitud de INT0 será atendida primero y a continuación la de UART Rx.
- Nota: al menos una instrucción del programa principal será ejecutada antes de atender el próximo pedido de interrupción (si hubiera pendiente).

Interrupciones anidadas

- Una interrupción NO puede interrumpir a otra interrupción que se este procesando.
- Sin embargo, si el programador lo desea, puede volver a habilitar la mascara I ($I=1$) dentro de una RSI para permitir este comportamiento.
- Esto se denomina [anidamiento de interrupciones](#).
- Una interrupcion en curso solo puede ser interrumpida por otra interrupción de mayor prioridad



- No es Recomendable el anidamiento de interrupciones.
- Es casi imposible crear código que se ajuste bien para todas las posibles combinaciones de interrupciones, aun siendo priorizadas.
- Se reduce la posibilidad de predecir el comportamiento del sistema en todas las posibles condiciones, es decir se pierde confiabilidad ya que no todas las condiciones son testeadas.

Latencia de una Interrupción

- Es el tiempo que tarda el Controlador de interrupciones en dar respuesta a una interrupción.
- Se mide desde que se recibe el pedido hasta que efectivamente se ejecuta la primer instrucción de la RSI correspondiente.
- En los AVR la latencia es de **4 ciclos de reloj** como mínimo.
 - Durante este tiempo, se guarda el PC en la pila, se pone I en 0 y se busca el vector de interrupción de mayor prioridad que corresponda.
- El retorno de la interrupción (RETI) tambien lleva 4 ciclos.
- En el caso en que el micro este en modo SLEEP, la latencia es de 8 ciclos.

La latencia de la interrupción también se ve afectada por:

- 1-la espera de que termine la instrucción en curso
- 2-otra/s interrupciones pendientes de mayor prioridad
- 3-el guardado de contexto que depende de cuantos registros CPU tenga que guardar.

Interrupciones y el compilador AVR-GCC

- Para utilizar interrupciones en un programa debemos agrandar el archivo de cabecera de Interrupciones:
 - `#include <avr\interrupt.h>`
- El código que queremos ejecutar en respuesta a una interrupción será RSI o ISR (Interrupt service routine en inglés):

```
ISR (interrupt vector name)  
{  
    //nuestro código  
}
```

- Notar que es una función especial que recibe como parámetro el nombre del vector de interrupción y no devuelve ningún argumento de salida.
- Para habilitar o deshabilitar la máscara global se utilizan los Macros `cli()`, `sei()`:
 - Son implementaciones equivalentes de las instrucciones de assembler

```
#define cli() __asm__ __volatile__ ("cli" ::)  
#define sei() __asm__ __volatile__ ("sei" ::)
```

Interrupciones y el compilador AVR-GCC

- **ISR(Vector Name):**

definidos en io.h



```
#define INT0_vect      _VECTOR(1)  /* External Interrupt Request 0 */
#define INT1_vect      _VECTOR(2)  /* External Interrupt Request 1 */
#define PCINT0_vect    _VECTOR(3)  /* Pin Change Interrupt Request 0 */
#define PCINT1_vect    _VECTOR(4)  /* Pin Change Interrupt Request 0 */
#define PCINT2_vect    _VECTOR(5)  /* Pin Change Interrupt Request 1 */
#define WDT_vect       _VECTOR(6)  /* Watchdog Time-out Interrupt */
#define TIMER2_COMPA_vect _VECTOR(7) /* Timer/Counter2 Compare Match A */
#define TIMER2_COMPB_vect _VECTOR(8) /* Timer/Counter2 Compare Match A */
#define TIMER2_OVF_vect _VECTOR(9)  /* Timer/Counter2 Overflow */
#define TIMER1_CAPT_vect _VECTOR(10) /* Timer/Counter1 Capture Event */
#define TIMER1_COMPA_vect _VECTOR(11) /* Timer/Counter1 Compare Match A */
#define TIMER1_COMPB_vect _VECTOR(12) /* Timer/Counter1 Compare Match B */
#define TIMER1_OVF_vect  _VECTOR(13) /* Timer/Counter1 Overflow */
```

Interrupciones y el compilador AVR-GCC

• **ISR(Vector Name):**

definidos en io.h



```
#define TIMER0_COMPA_vect _VECTOR(14) /* TimerCounter0 Compare Match A */
#define TIMER0_COMPB_vect _VECTOR(15) /* TimerCounter0 Compare Match B */
#define TIMER0_OVF_vect   _VECTOR(16) /* Timer/Couner0 Overflow */
#define SPI_STC_vect       _VECTOR(17) /* SPI Serial Transfer Complete */
#define USART_RX_vect      _VECTOR(18) /* USART Rx Complete */
#define USART_UDRE_vect    _VECTOR(19) /* USART, Data Register Empty */
#define USART_TX_vect      _VECTOR(20) /* USART Tx Complete */
#define ADC_vect           _VECTOR(21) /* ADC Conversion Complete */
#define EE_READY_vect      _VECTOR(22) /* EEPROM Ready */
#define ANALOG_COMP_vect   _VECTOR(23) /* Analog Comparator */
#define TWI_vect           _VECTOR(24) /* Two-wire Serial Interface */
#define SPM_READY_vect     _VECTOR(25) /* Store Program Memory Read */
```

Interrupciones y el compilador AVR-GCC

```
#include <avr/io.h>
#include <avr/interrupt.h>
```

```
int main (void) {
    DDRB |= (1<<DDRB0);
    TIMSK=(1<<TOIE0);
    sei();
    while(1) {;}

    return 0;
}
```

```
ISR (TIMER0_OVF_vect)
{
    PORTB |= (1<<PORTB0);
}
```

Ejemplo de programa usando interrupciones

Assembler de la ISR:

```
ISR (TIMER0_OVF_vect)
{
    push    r0                ;r0->(SP)
    in      r0, SREG
    push    r0                ;SREG->(SP)

    PORTA |= (1<<PA0);
    sbi     PORTA, 0

    pop     r0
    out     SREG, r0          ;(SP)->SREG
    pop     r0                ;(SP)->r0
    reti

}
```

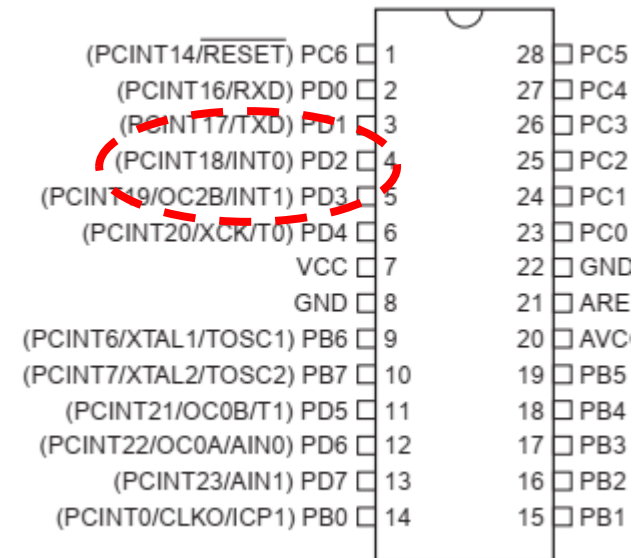
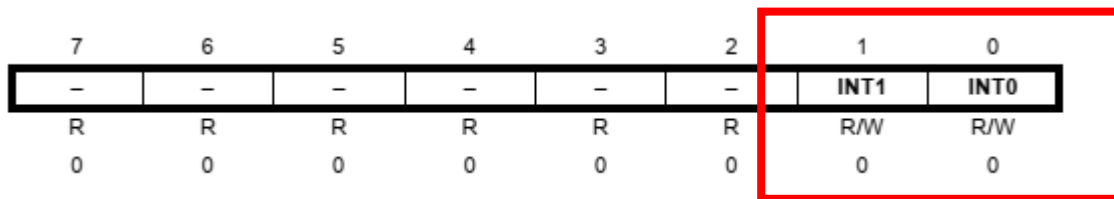
Interrupciones y el compilador AVR-GCC

- **Importante:**

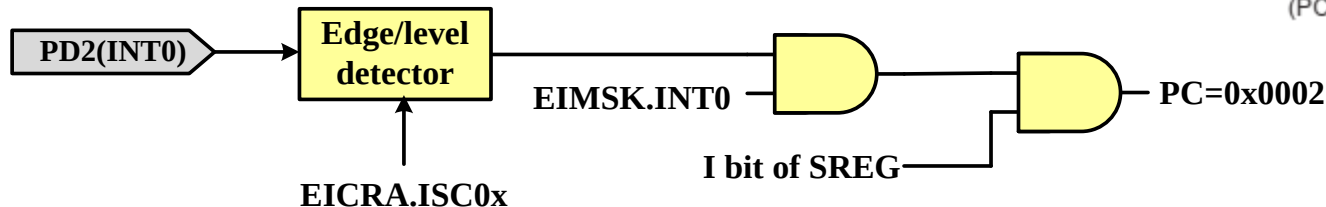
- El compilador, automáticamente agrega instrucciones al principio y al final de cada RSI para guardar y restablecer el contexto de TODOS los registros CPU y del SREG en la pila.
- No es posible pasar parámetros a una RSI. Se deben utilizar **variables globales**, especificadas como ***volatile*** para compartir información con otra parte del programa.
- ***Volatile*** indica al compilador que no es posible saber cuando la variable puede cambiar su valor (*Ejemplo: los puertos I/O o los flags de estado de los periféricos*) y por lo tanto no debe aplicar optimizaciones sobre la misma.
- **Ejemplo del porqué: pág 126 en Libro de Espinosa.**

Las Interrupciones Externas

- Hay dos terminales INT0(PD2), INT1(PD3) en el ATMEGA328p que pueden generar interrupciones de periféricos externos
- Se habilitan con el registro **EIMSK – External Interrupt Mask Register**



INT0 = 1 Enable Int
INT0 = 0 Disable Int



INT0 o INT1 son "activas en nivel bajo" por defecto.

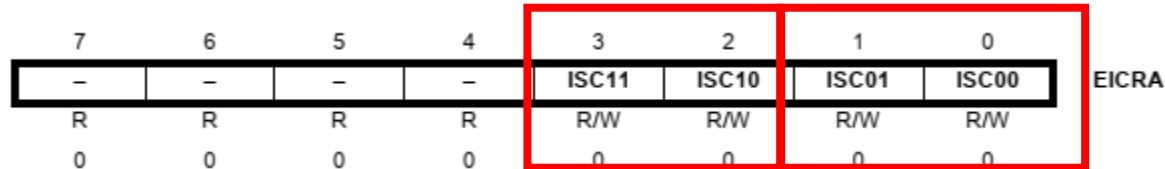
Tabla de Vectores- ATMEGA

Table 12-6. Reset and Interrupt Vectors in ATmega328 and ATmega328P

VectorNo.	Program Address ⁽²⁾	Source	Interrupt Definition
1	0x0000 ⁽¹⁾	RESET	External Pin, Power-on Reset, Brown-out Reset and Watchdog System Reset
2	0x0002	INT0	External Interrupt Request 0
3	0x0004	INT1	External Interrupt Request 1
4	0x0006	PCINT0	Pin Change Interrupt Request 0
5	0x0008	PCINT1	Pin Change Interrupt Request 1
6	0x000A	PCINT2	Pin Change Interrupt Request 2
7	0x000C	WDT	Watchdog Time-out Interrupt
8	0x000E	TIMER2_COMPA	Timer/Counter2 Compare Match A
9	0x0010	TIMER2_COMPB	Timer/Counter2 Compare Match B
10	0x0012	TIMER2_OVF	Timer/Counter2 Overflow
11	0x0014	TIMER1_CAPT	Timer/Counter1 Capture Event
12	0x0016	TIMER1_COMPA	Timer/Counter1 Compare Match A
13	0x0018	TIMER1_COMPB	Timer/Counter1 Compare Match B
14	0x001A	TIMER1_OVF	Timer/Counter1 Overflow
15	0x001C	TIMER0_COMPA	Timer/Counter0 Compare Match A
16	0x001E	TIMER0_COMPB	Timer/Counter0 Compare Match B
17	0x0020	TIMER0_OVF	Timer/Counter0 Overflow
18	0x0022	SPI_STC	SPI Serial Transfer Complete
19	0x0024	USART_RX	USART Rx Complete
20	0x0026	USART_UDRE	USART, Data Register Empty
21	0x0028	USART_TX	USART, Tx Complete
22	0x002A	ADC	ADC Conversion Complete
23	0x002C	EE_READY	EEPROM Ready
24	0x002E	ANALOG_COMP	Analog Comparator
25	0x0030	TWI	2-wire Serial Interface
26	0x0032	SPM_Ready	Store Program Memory Ready

Las Interrupciones Externas

- El tipo de activación de las interrupciones INT0 e INT1 se puede configurar en el registro **EICRA**: External Interrupt Control Register



ISCx1	ISCx0	
0	0	
0	1	
1	0	
1	1	

Las Interrupciones Externas

- Las interrupciones son registradas en un Flag y el mecanismo de reconocimiento es automático



- Las bandera se limpia automáticamente por hardware cuando se concluye con la atención a la interrupción.
- Alternativamente, se pueden borrar escribiendo un 1 en el flag correspondiente.

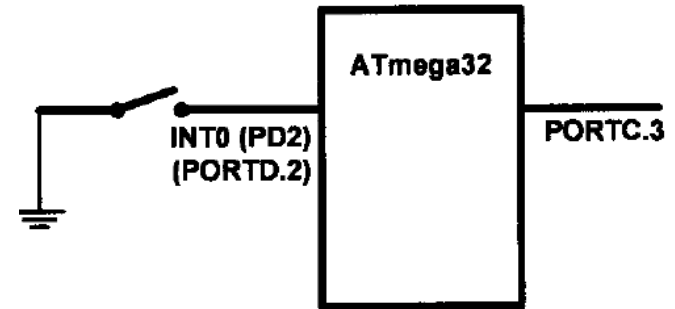
Las Interrupciones Externas

- **Ejemplo:** Un nivel bajo en INT0 interrumpe para negar el bit 3 de PORTC

```
#include <avr/io.h>
#include <avr/interrupt.h>

int main(void)
{
    DDRC |= (1<<DDC3);
    PORTD |= (1<<PORTD2);
    EIMSK |= (1<<INT0); //Habilitar int INT0
    sei();               //Habilitar int globales
    while (1) ;
}

ISR(INT0_vect){
    PORTC ^= (1<<PORTC3);
}
```



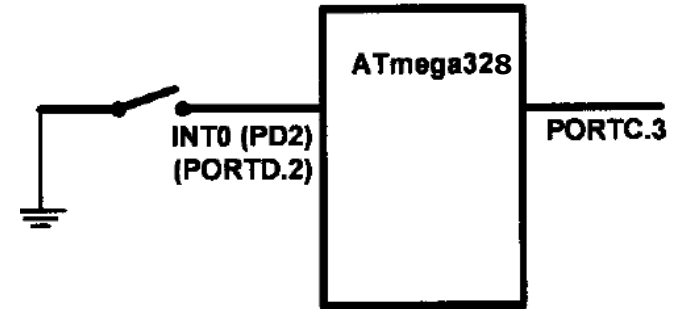
Las Interrupciones Externas

- **Ejemplo:** Flanco de bajada en INT0 interrumpe para negar bit 3 de PORTC

```
#include <avr/io.h>
#include <avr/interrupt.h>

int main(void)
{
    DDRC |= (1<<DDC3);
    PORTD |= (1<<PORTD2);
    EICRA |= (1<<ISC01); //INT0 x flanco de bajada
    EIMSK |= (1<<INT0);  //Habilitar int INT0
    sei();                //Habilitar int globales
    while (1) ;
}

ISR(INT0_vect){
    PORTC ^= (1<<PORTC3);
}
```



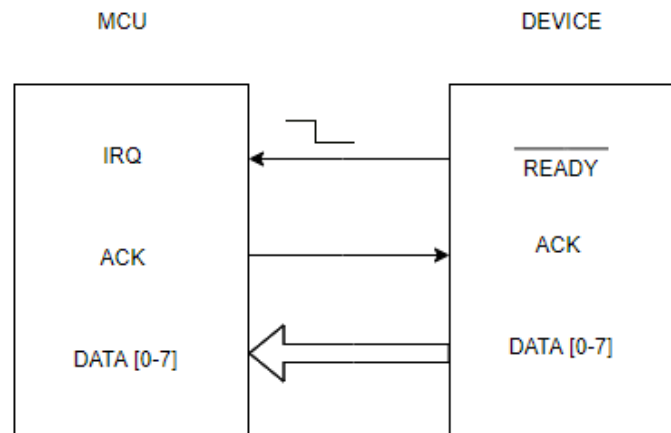
Tarea para la casa:

Copiar y ejecutar los ejemplos en el simulador.

Modificar el programa del TP1 para cambiar la secuencia de encendido de LEDS utilizando un pulsador conectado a una entrada de interrupción externa.

¿Interrupciones por nivel o por flanco?

- Si una interrupción funciona por nivel quiere decir que el periférico que la genera “**coloca y mantiene**” el nivel en la línea para que el uC atienda a esa petición. Durante la atención, el uC debería indicar al periférico externo, de algún modo, que ha sido atendido para que éste libere el nivel de la línea.
- Notar que al no ser una “**petición registrada**”, si el nivel bajo no está presente cuando las interrupciones están habilitadas, el pedido no será tenido en cuenta.
- Por otro lado, si el periférico no retira el nivel de la línea, continuará solicitando interrupción indefinidamente.
- Por lo tanto, las interrupciones **por nivel**, no tienen memoria, y requieren de un aviso al periférico para que no se procese la misma interrupción múltiples veces.



¿Interrupciones por nivel o por flanco?

- Si una interrupción funciona por flanco quiere decir que el periférico produce un flanco en la línea y este pedido queda **registrado en un Flag (Flip Flop)** pidiendo interrupción. Típicamente el uC borra este flag para indicar que esta interrupción ya ha sido atendida sin necesidad de comunicárselo al periférico.
- De esta manera, si las interrupciones están deshabilitas al momento de producirse el flanco, los pedidos quedan **“pendientes”** y serán atendidos por prioridad cuando se active la máscara de interrupción I.

